

Fitbauer: an open-source, NORMOS-compatible Python program for ^{57}Fe Mössbauer spectral fitting

Jorge Sánchez Marcos · Nieves Menéndez González

Received: October 9, 2026

Abstract We present **Fitbauer**, an open-source desktop program written in Python for folding, simulating and least-squares fitting ^{57}Fe Mössbauer transmission spectra. Fitbauer is modelled on NORMOS, the Fortran package that has been a reference in the field for more than three decades: it adopts its physical model, parameterisation, folding procedure and constraint scheme, and it reads and writes NORMOS job (.JOB) files, so that existing analyses can be carried over directly. It is, however, an independent implementation, not a translation of the Fortran code, and it introduces a few numerical improvements (exact Voigt profiles, double-precision diagonalisation of the hyperfine Hamiltonian, sub-channel folding interpolation, correct uncertainties for tied parameters). The program offers discrete fitting of up to ten components (singlets, doublets, sextets and magnetic-relaxation lineshapes), hyperfine-field and quadrupole-splitting distributions with regularisation, and three uncertainty estimates (covariance, parametric bootstrap and profile likelihood), all driven by a live simulation of the direct model. Its agreement with NORMOS has been verified on more than 400 synthetic spectra generated by NORMOS and on 564 fits of real spectra originally performed with NORMOS; on four reference compounds the hyperfine fields agree within 0.012 T and the isomer shifts within 0.004 mm s^{-1} . Fitbauer runs on Windows, Linux and macOS, offers its interface in eight languages and is distributed under the Apache 2.0 licence.

Keywords Mössbauer spectroscopy · Spectral fitting · NORMOS · Hyperfine parameters · Distribution analysis · Open source · Python

J. Sánchez Marcos · N. Menéndez González
Departamento de Química Física, Facultad de Ciencias, Universidad Autónoma de Madrid,
Campus de Cantoblanco, 28049 Madrid, Spain
E-mail: jorge.sanchezm@uam.es

1 Introduction

^{57}Fe Mössbauer spectroscopy remains one of the most direct probes of local electronic structure, magnetic order and oxidation state in iron-containing compounds. Its widespread use in mineralogy, environmental science, materials science and geochemistry depends critically on the quality of the software used to extract hyperfine parameters from the data [1,2].

Several programs have been developed over the past four decades. NORMOS [3,4], with its SITE (discrete sites) and DIST (distributions) modules, is among the most widely used and cited. It is written in Fortran, runs under MS-DOS and is no longer actively developed; running it today requires an emulator. Yet many laboratories hold years of analyses stored as NORMOS job files, and its results are a de facto reference. MossWinn [5] is a comprehensive commercial package for Windows. SpectrRelax [6] handles relaxation and distribution models on Windows. Moessfit [7] is a free Qt/C++ program focused on simultaneous multi-dataset fitting. RECOIL [8] and VINDA [9] are further established options. None of these programs is open source, which limits the reproducibility of published results and makes independent auditing of the code impossible.

Fitbauer was written to offer a modern, open and cross-platform alternative that *behaves like NORMOS*: a user coming from NORMOS finds the same model, the same parameters and the same conventions, can open his or her own job files, and obtains the same results. Three further goals guided the design: (i) complete separation of the physics engine from the graphical interface, so that every analysis can also be run from scripts or the command line; (ii) a framework that makes it straightforward to add new models; (iii) systematic testing, both internal and against NORMOS itself.

Section 2 describes the physical model and its relation to NORMOS. Section 3 outlines the software architecture and how it can be extended. Section 4 summarises the main features. Section 5 presents the validation, including the comparison with NORMOS on many spectra. Section 6 shows four reference fits. Section 7 discusses advantages and limitations, and Section 8 concludes.

2 Physical model

2.1 Relation to NORMOS

Fitbauer's physics was written independently in Python and later reviewed, part by part, against both the NORMOS executable and its Fortran source. That review served to verify the model and to adopt NORMOS's conventions where they matter to the user: the definition of the parameters (line positions, widths and area or depth ratios), the form of the linear constraints between parameters, the two-cycle search of the folding point, and the smoothing matrix used in distributions and its edge anchors. Where Fitbauer deliberately departs from NORMOS it does so to be more exact, and the differences are documented (Section 7).

2.2 Lineshape and transmission

In the thin-absorber approximation the transmission spectrum is

$$T(v) = B(v) - \sum_{k=1}^N A_k(v), \quad (1)$$

where v is the source velocity, $A_k(v)$ is the absorption of the k -th component and $B(v)$ is the background: a constant, optionally with linear to quartic terms in v (as the background polynomial of NORMOS). Each line is a Lorentzian of full width at half maximum Γ ,

$$L(v; v_0, \Gamma) = \frac{(\Gamma/2)^2}{(v - v_0)^2 + (\Gamma/2)^2}, \quad (2)$$

or, optionally, an exact Voigt profile (convolution with a Gaussian of width σ) evaluated through the Faddeeva function.

For optically thick samples two models are available: the saturation form of Margulies and Ehrman [10], $T = B - C(1 - e^{-A_{\text{tot}}/C})$ with C a saturation scale, and the full transmission integral, in which the source Lorentzian is convolved with $1 - e^{-\tau(v)}$ and an optional resonant fraction of the source is fitted, as in NORMOS.

Spectra recorded with a triangular velocity drive are folded around a folding point found automatically by a two-cycle symmetry search, as in NORMOS; Fitbauer interpolates sub-channel positions with a cubic spline, so that a folding point that is not a half-integer does not broaden the lines. Sinusoidal-drive data (NORMOS TRIANG=.FALSE.) are fitted unfolded, each channel at its true velocity.

2.3 Discrete components

Up to ten components can be superimposed. Each one is a singlet, a doublet, a sextet or one of three magnetic-relaxation lineshapes. Line intensities follow the 3:2:1 powder pattern or are fitted freely. For sextets with a small quadrupole interaction the usual first-order shift $\pm \Delta E_Q/2$ is applied; when the quadrupole interaction is not small, the static hyperfine Hamiltonian (magnetic field plus an electric field gradient of arbitrary asymmetry η) is diagonalised exactly in double precision [11], and the line intensities are computed from the eigenvectors, either as a powder average or for a single crystal with a given beam direction.

The relaxation lineshapes are a phenomenological interpolation between a blocked sextet and a superparamagnetic doublet, the two-state exchange model of Blume and Tjon [12] (the model NORMOS uses for Ising relaxation, with optionally unequal populations of the two states), and a Néel–Arrhenius model [13, 14] for polydisperse nanoparticle assemblies.

Any parameter can be fixed, left free or tied to another one by a linear constraint $p_{\text{target}} = f p_{\text{source}} + c$, which is exactly the form of the NORMOS constraints.

2.4 Hyperfine field and quadrupole-splitting distributions

For magnetically disordered or nanostructured samples Fitbauer fits distributions $P(B_{\text{hf}})$ and $P(\Delta E_Q)$ following the Hesse–Rübartsch method [15]: a sum of many sextets (or doublets) on a uniform grid, with non-negative amplitudes regularised by minimising

$$F = \|\mathbf{y} - \mathbf{A}\mathbf{p}\|_w^2 + \alpha \|\mathbf{D}^2\mathbf{p}\|^2, \quad \mathbf{p} \geq 0, \quad (3)$$

where $\|\cdot\|_w$ is the weighted norm and $\mathbf{D}^2$ the second-difference matrix; the resulting smoothing matrix is identical to that of NORMOS-DIST, including its optional edge anchors. Total-variation and maximum-entropy regularisers are also available. The regularisation parameter α is chosen with the L-curve criterion [16], which can be inspected and overridden by clicking on the curve. Sharp (discrete) components can be fitted simultaneously with the distribution, and parametric shapes (Gaussian, binomial and the Voigt-based sum of Gaussians of Rancourt and Ping [17]) can replace the free histogram.

2.5 Velocity calibration

By default the velocity scale is calibrated with an α -Fe foil, $B_{\text{hf}} \equiv 33.0$ T, using the published experimental line positions of α -Fe [18] ($\pm 0.839/\pm 3.084/\pm 5.329$ mm s⁻¹). NORMOS instead derives the line positions from nuclear moments; the two patterns differ slightly in the ratio of the inner to the outer lines, so that the same spectrum yields fields that differ by about 0.01 T. Both conventions can be selected, which allows NORMOS results to be reproduced exactly when required. Once determined, the calibration is stored and reused for every subsequent spectrum recorded with the same set-up.

3 Software architecture and extensibility

All the physics and optimisation code lives in the `core/` package, which does not depend on any graphical toolkit. The Qt interface (PySide6 with Matplotlib) and the command-line tools are thin clients of `core/`, so the same calculation can be run interactively, from a Python script or from a batch pipeline. The main modules are:

- `core/physics.py`, `core/hamiltonian.py` Lineshapes, component absorption and the hyperfine Hamiltonian, written as pure functions.
- `core/params.py` A single registry of the model parameters (names, default values, bounds and which component types use them).
- `core/fit_engine.py` Bounded least-squares optimisation (`scipy.optimize.least_squares`), multi-start, constraints and uncertainties.
- `core/folding.py`, `core/data_io.py` Spectrum input and folding.
- `core/normos_job.py` Reading and writing NORMOS job files.

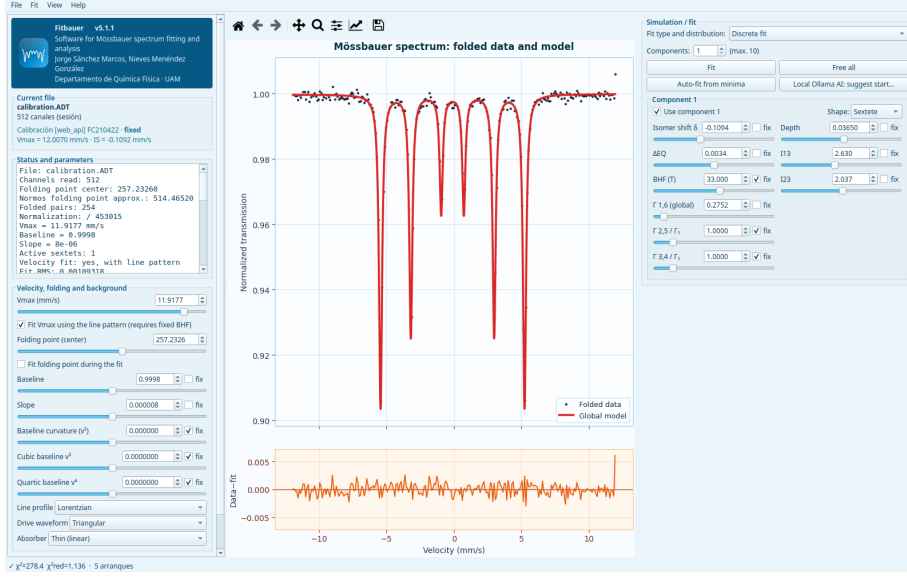


Fig. 1 Fitbauer main window with a fitted α -Fe calibration spectrum. Left: file status, calibration, folding and background controls. Centre: data, model and residuals. Right: simulation and fit panel with the parameters of the active component.

`core/session.py` Headless orchestration (load $\rightarrow$ fold $\rightarrow$ fit $\rightarrow$ export) used by the command-line tools.

`mossbauer_distribution.py` Distribution fitting.

This layout is what makes the program easy to extend. A new lineshape or model is added as a pure function in `core/`, its parameters are declared once in the registry, and from there it becomes available to the fit engine, the interface, saved sessions and the command line. Several of the capabilities described above (the full Hamiltonian, the transmission integral, single-crystal geometry, unequal relaxation populations) were added in this way, each with its own tests, without changing the rest of the program.

Figs. 1 and 2 show the main window and a discrete fit.

4 Main features

4.1 Live simulation of the direct model

The direct model is always *live*: every change to a physical parameter (δ , ΔE_Q , B_{hf} , widths, depths, intensities) redraws the spectrum and its subspectra at once. A physically sensible starting model can thus be built by eye before fitting, which makes the non-linear fit faster and less prone to spurious minima; the effect of each parameter can be explored interactively, which is useful for

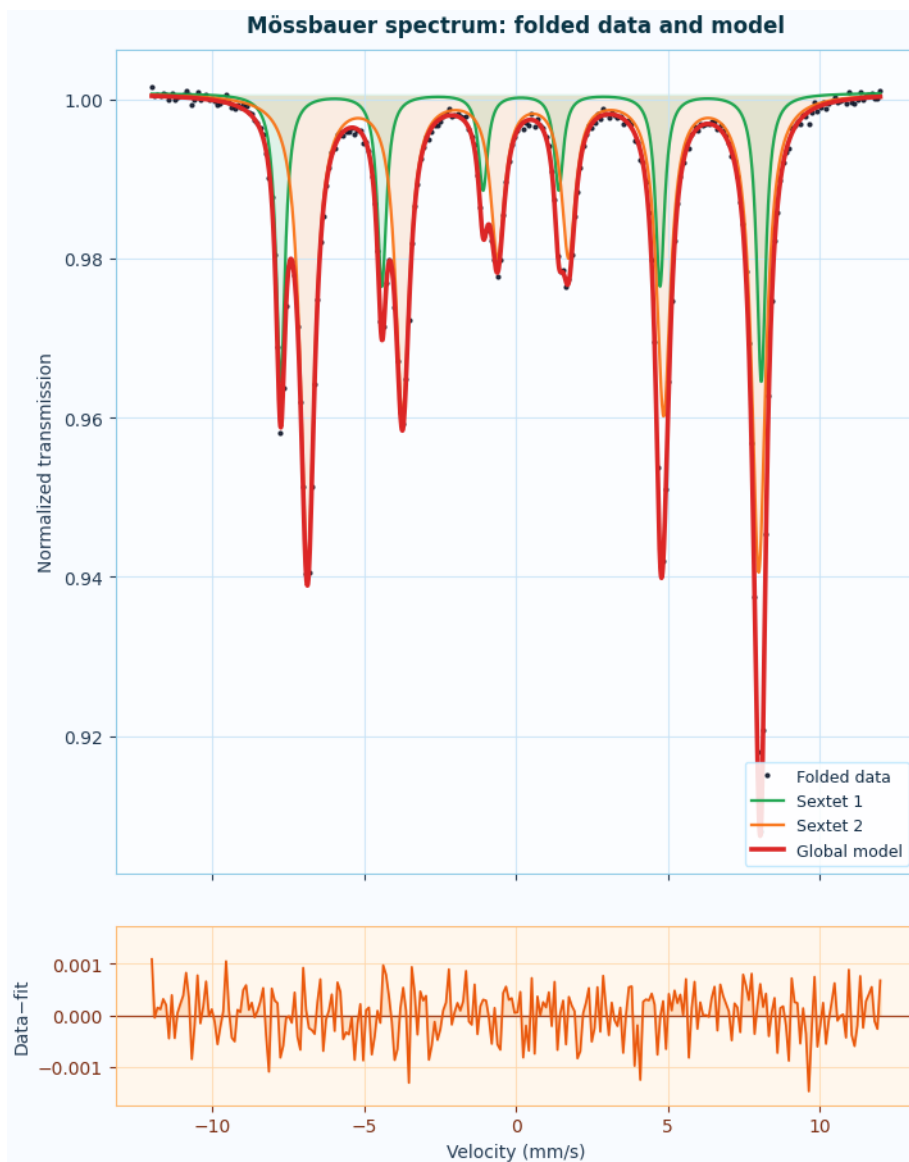


Fig. 2 Discrete fit of a two-component spectrum (magnetite Fe_3O_4). The two sextet subspectra are shown individually with optional area shading; the total model is the thick curve and the residuals are shown in the lower panel.

teaching; and hypotheses can be compared with the data without fitting anything. Simulation and fitting share exactly the same model. Starting values can also be proposed automatically from the absorption minima detected in the spectrum.



Fig. 3 Distribution fit of a synthetic spectrum generated from a bimodal $P(B_{\text{hf}})$ at 28 and 48 T. Left: spectrum and residuals. Right: the reconstructed $P(B_{\text{hf}})$, which recovers both populations.

4.2 Fitting

In *discrete mode* the user chooses up to ten components and fixes, frees or ties each parameter. The fit uses a multi-start strategy (eight restarts around the starting point) and switches automatically to a global search (differential evolution followed by local refinement) when the local fits remain poor. The noise model can be Gaussian or Poissonian. In *distribution mode* (Fig. 3) the program computes $P(B_{\text{hf}})$ or $P(\Delta E_Q)$ on a user-defined grid, optionally with sharp components, and the regularisation parameter is selected with the L-curve or set by hand.

4.3 Uncertainties

Three estimates can be computed for the same fit:

1. *Covariance matrix* (default): 1σ errors from the Jacobian at the minimum, $\sigma_i = \sqrt{(J^\top W J)_{ii}^{-1}}$. Since the counting statistics of each channel are known, these errors are not rescaled by the reduced χ^2 , as in NORMOS. The error of a tied parameter is propagated from its source parameter.
2. *Parametric bootstrap*: N synthetic spectra are generated from the fitted model with Poisson (or Gaussian) noise and refitted; the spread of the results gives asymmetric percentile intervals.

3. *Profile likelihood*: the parameter of interest is stepped away from its best-fit value while all the others are re-optimised; the 1σ and 2σ intervals are given by $\Delta\chi^2 = 1$ and 4 [19].

4.4 Files, sessions and other tools

Fitbauer reads WissEl `.ws5` files, single-column ASCII (`.adt`) count files and two-column velocity–transmission text files, and it imports and exports NORMOS job files for both SITE and DIST, loading the spectrum the job refers to. A job exported by Fitbauer is accepted by NORMOS, which reproduces the same theoretical spectrum. A session file stores the complete state of an analysis (data, calibration, model and results), so that any fit can be reopened and reproduced. Further tools include batch fitting of spectrum series with warm start from the previous result, report export, and command-line programs that cover the same fits as the interface. The interface is available in eight languages (Spanish, English, French, German, Portuguese, Japanese, Russian and Chinese).

5 Validation

5.1 Internal test suite

Fitbauer is distributed with an automated suite of 587 tests covering the physics, folding, the fit engine, distributions, sessions, the command-line tools and the graphical interface. One module, `tests/test_synthetic_validation.py`, is organised in six levels of increasing rigour (Table 1):

- *Self-consistency*: noise-free spectra generated with a reference implementation that shares no code with `core/` are fitted back, and must be recovered to $\tilde{\chi}^2 < 10^{-6}$.
- *Jacobian*: the analytical derivatives are compared with finite differences (1 % tolerance).
- *Monte Carlo pull statistics*: $N = 200$ spectra per component type with Poisson noise; the pulls $z_j = (\hat{\theta}_j - \theta_j^{\text{true}})/\sigma_j$ must satisfy $|\bar{z}| < 0.3$ and $\sigma(z) \in (0.5, 2.0)$. This is the test that checks that the reported error bars are neither too small nor too large.
- *Hard cases, physical constraints and real data*: overlapping lines, narrow doublets, low signal-to-noise, bounds, and the fit of the included α -Fe spectrum.

5.2 Comparison with NORMOS on many spectra

The central question for a user coming from NORMOS is whether both programs give the same answer. We addressed it in three ways, using NORMOS/SITE

Table 1 Levels of the synthetic validation module.

Level	Type	Tests	Approx. time
1	Self-consistency	4	< 1 s
2	Jacobian	5	< 1 s
3	Monte Carlo pulls	3	≈ 46 s
4	Hard cases	8	< 1 s
5	Physical constraints	9	< 1 s
6	Real data (α -Fe)	4	≈ 1 s
Total		34	≈ 48 s

and NORMOS/DIST (demonstration version 27.01.1994, WissEl GmbH) run under the DOSBox emulator.

Synthetic bank. NORMOS was used to generate more than 400 spectra from known parameters, covering the cases its manual describes: singlets, doublets and sextets over wide ranges of δ , ΔE_Q , B_{hf} and Γ , texture, up to ten sites with constraints, thick absorbers, background shapes, different numbers of channels and velocity ranges, the full Hamiltonian and distributions. Fitbauer fitted each spectrum (about 6 500 parameter comparisons). For the first-order model the median deviation from the true values is $2 \times 10^{-7} \text{ mm s}^{-1}$ in line positions and $4 \times 10^{-5} \text{ T}$ in B_{hf} , i.e. the two programs compute the same spectrum; with the full Hamiltonian the medians are $6 \times 10^{-5} \text{ mm s}^{-1}$ and $8 \times 10^{-4} \text{ T}$. The whole bank is regenerated by a single script; rerunning it with the current version reproduces these figures.

Relaxation. SITE’s Ising relaxation was probed separately over six decades of the relaxation rate and for several degrees of polarization. Fitbauer’s two-state model reproduces the NORMOS curves to 10^{-5} of the peak without fitting the rate, which confirms that NORMOS’s rate parameter `OME` (in MHz) equals twice Fitbauer’s exchange rate. For unequal populations NORMOS uses a rule that does not satisfy detailed balance; Fitbauer offers it as an optional “NORMOS convention” and keeps the physical model as the default.

Source-level review. The NORMOS Fortran source was compared, routine by routine, with Fitbauer’s `core/`: constants and line pattern, lineshape, folding, transmission integral, statistics, the Hamiltonian, relaxation and distributions. The review confirmed that the models are equivalent and identified the small differences discussed in Section 7. The NORMOS source is proprietary and is not part of Fitbauer.

Real analyses. Finally, 564 fits of real laboratory spectra that had been performed with NORMOS over the years were reloaded from their job files and refitted with Fitbauer. The fitted parameters agree with those of NORMOS to 0.0011 mm s^{-1} in δ , 0.0019 mm s^{-1} in ΔE_Q , 0.017 T in B_{hf} and 0.030 mm s^{-1} in Γ , and in 355 of the 503 comparable jobs (71 %) Fitbauer reaches the same or a lower reduced χ^2 .

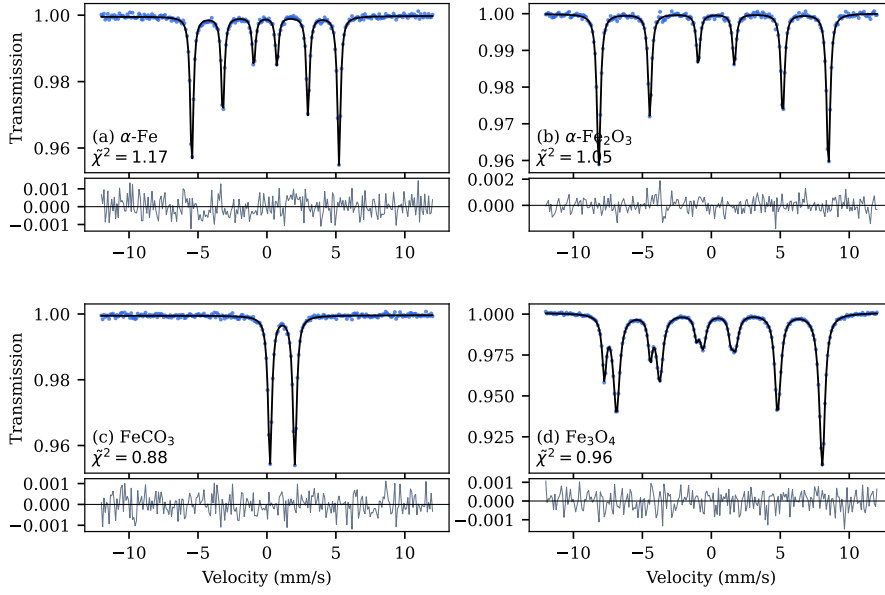


Fig. 4 Fitbauer fits (black) to four reference spectra (points), with residuals below each panel: (a) α -Fe, (b) hematite, (c) siderite, (d) magnetite.

6 Application examples

We illustrate the results on four reference spectra included with the program: α -Fe, hematite (α - Fe_2O_3), siderite (FeCO_3) and magnetite (Fe_3O_4). All consist of 512 channels recorded with a triangular drive and $V_{\text{max}} \approx 12.0 \text{ mm s}^{-1}$, and were fitted with a single sextet (α -Fe, hematite), a doublet (siderite) and two sextets (magnetite). Fig. 4 shows the fits.

The same four spectra were fitted with NORMOS/SITE starting from the same models. Table 2 and Fig. 5 compare the results. The largest differences are 0.012 T in B_{hf} , 0.004 mm s^{-1} in δ , 0.001 mm s^{-1} in $|\Delta E_Q|$ and 0.007 mm s^{-1} in Γ , comparable to the statistical uncertainties of the fits. They arise from the different folding (NORMOS sums whole channels around its folding point, Fitbauer interpolates) and from the background model (constant in NORMOS, with a linear slope in Fitbauer). The reduced χ^2 values are also close; they are not strictly comparable because NORMOS normalises by the data and uses one degree of freedom less. For siderite the sign of ΔE_Q differs only by convention: both programs give the same line positions.

Table 2 Fitbauer and NORMOS/SITE results on the same spectra. δ_{raw} is relative to the source (not corrected to α -Fe). Errors are 1σ .

Compound	Program	B_{hf} (T)	δ_{raw} (mm/s)	$ \Delta E_Q $ (mm/s)	Γ (mm/s)	$\tilde{\chi}^2$
α -Fe	NORMOS	33.035(6)	−0.110(1)	—	0.279(2)	1.195
	Fitbauer	33.046(6)	−0.110(1)	—	0.278(2)	1.165
α -Fe ₂ O ₃	NORMOS	51.576(7)	0.262(1)	0.200(2)	0.317(3)	1.058
	Fitbauer	51.581(7)	0.261(1)	0.199(2)	0.318(3)	1.054
FeCO ₃	NORMOS	—	1.121(1)	1.797(2)	0.337(3)	0.883
	Fitbauer	—	1.122(1)	1.798(2)	0.338(3)	0.878
Fe ₃ O ₄ (A)	NORMOS	49.075(10)	0.159(1)	—	0.368(6)	1.093
	Fitbauer	49.070(10)	0.163(1)	—	0.362(4)	0.959
Fe ₃ O ₄ (B)	NORMOS	46.079(7)	0.564(1)	—	0.549(3)	
	Fitbauer	46.067(8)	0.562(1)	—	0.556(3)	

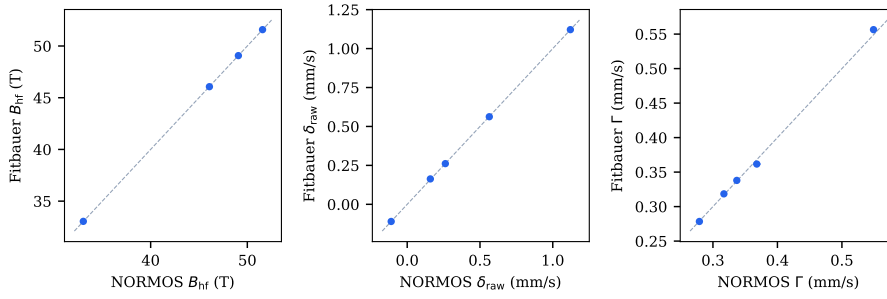


Fig. 5 Fitbauer versus NORMOS for the hyperfine field (left), the isomer shift (centre) and the line width (right) of the four reference spectra (magnetite contributes two sites). Error bars are 1σ ; the dashed line is $y = x$.

7 Discussion

7.1 Differences from NORMOS

Fitbauer reproduces NORMOS where NORMOS is exact, and departs from it in a few places where a more accurate calculation is now inexpensive:

- Voigt lines are computed exactly from the Faddeeva function rather than with the pseudo-Voigt approximation used by SITE.
- The hyperfine Hamiltonian is diagonalised with a Hermitian solver in double precision. SITE uses a general complex solver in single precision, whose eigenvectors are not orthonormalised; its deviation grows with the mixing of states.
- Folding interpolates sub-channel positions, so that the folding point need not be a half-integer and lines are not broadened.

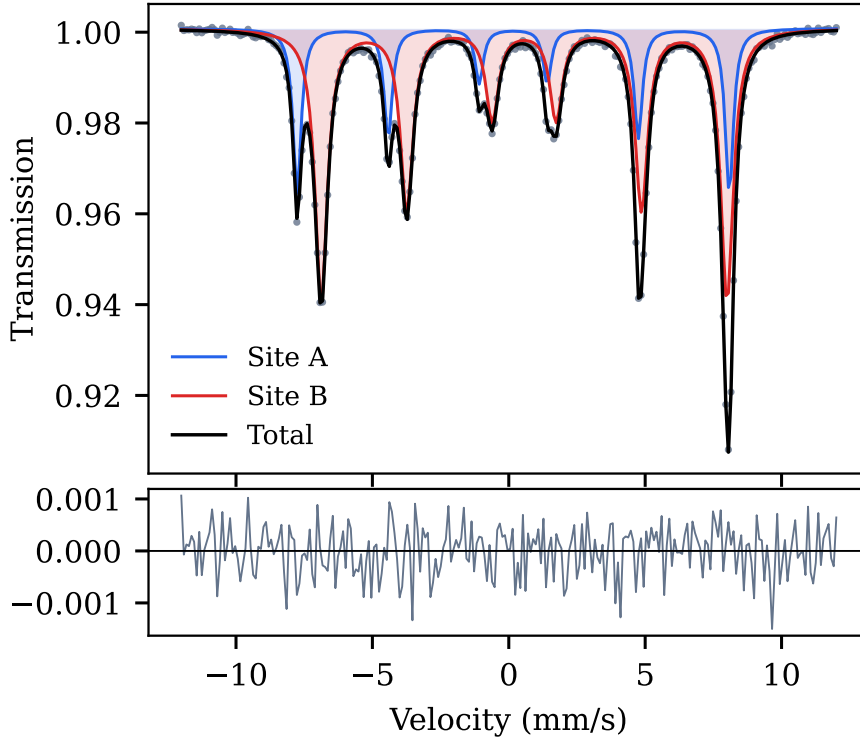


Fig. 6 Two-sextet fit of magnetite. Site A (tetrahedral Fe^{3+}) and site B (octahedral $\text{Fe}^{2.5+}$) are shown separately; the total model is in black. Relative to $\alpha\text{-Fe}$: $\delta_{\text{A}} = 0.273 \text{ mm s}^{-1}$, $B_{\text{hf}}^{\text{A}} = 49.1 \text{ T}$; $\delta_{\text{B}} = 0.672 \text{ mm s}^{-1}$, $B_{\text{hf}}^{\text{B}} = 46.1 \text{ T}$; $\chi^2 = 0.96$.

- The uncertainty of a tied parameter is propagated with the multiplicative factor of the constraint. In NORMOS the additive constant is used instead, so that a purely multiplicative constraint is reported with zero error.

Each of these differences is small for typical spectra, as the comparisons of Sections 5.2 and 6 show.

7.2 Advantages

Being open source and built on widely used Python packages (NumPy, SciPy, Matplotlib, PySide6), Fitbauer can be installed on any current operating system and its computational environment reproduced exactly, which matters for the reproducibility of published results [20]. NORMOS users can migrate without redoing their work, thanks to the job-file compatibility. The program adds profile-likelihood and bootstrap uncertainties, a live simulation of the model and a scriptable interface.

Table 3 Feature comparison with existing Mössbauer fitting programs. ✓: available; ○: limited; —: not available.

Feature	Fitbauer	NORMOS	MossWinn	RECOIL
Open source	✓	—	—	—
Cross-platform	✓	—	—	—
NORMOS job files	✓	✓	—	—
$P(B_{\text{hf}})$ dist.	✓	✓	✓	○
Sharp + distribution	✓	✓	✓	—
Full Hamiltonian	✓	✓	✓	—
Two-state relaxation	✓	✓	✓	—
Thick absorber	✓	✓	✓	✓
Profile likelihood	✓	—	—	—
Bootstrap errors	✓	—	—	—
Automated test suite	✓	—	—	—
CLI / scripting	✓	○	—	—
Multilingual GUI	✓	—	○	—
Free of charge	✓	—	—	✓

7.3 Current limitations

Some NORMOS-DIST options are not yet available, notably the Czjzek (Le Caër) and Billard–Chamberod distributions. Multipolar M1+E2 mixing, irrelevant for ^{57}Fe , is not implemented, and neither are other Mössbauer isotopes. Conversion-electron and backscattering geometries, NORMOS’s spherical relaxation model and relaxation between more than two states are also absent. Thanks to the modular design of Section 3, these are natural targets for future development.

7.4 Comparison summary

Table 3 summarises the comparison with the most commonly used alternatives.

8 Conclusions

Fitbauer is an open-source Python program for ^{57}Fe Mössbauer spectral fitting that follows the physical model and conventions of NORMOS, reads and writes its job files, and gives the same results: on more than 400 synthetic spectra generated by NORMOS, on 564 real analyses originally performed with it, and on four reference compounds, where the hyperfine fields agree within 0.012 T and the isomer shifts within 0.004 mm s^{-1} . On this common basis it adds a live simulation of the model, a modern cross-platform interface in eight languages, command-line and scripting access, three uncertainty estimates and an extensive automated test suite, and its modular design makes new models straightforward to add.

The program, example spectra, calibration templates and a user manual are freely available at <https://github.com/sullymike/Fitbauer> and archived at Zenodo [21].

Code availability

Fitbauer is released under the Apache 2.0 licence; the Qt libraries it uses through PySide6 are distributed under the LGPLv3. Source code: <https://github.com/sullymike/Fitbauer>; archived version: doi:10.5281/zenodo.21858360. The reference spectra of Section 6 and the script that produces Figs. 4–6 and Table 2 (`docs/article_figures.py`) are included in the repository. The real NORMOS analyses of Section 5.2 are laboratory data and are not distributed.

Acknowledgements J.S.M. acknowledges the Departamento de Química Física (UAM) for computational resources and support.

References

1. N. N. Greenwood and T. C. Gibb. *Mössbauer Spectroscopy*. Chapman and Hall, London, 1971.
2. P. Gütlich, E. Bill, and A. X. Trautwein, editors. *Mössbauer Spectroscopy and Transition Metal Chemistry*. Springer, Berlin, 2011.
3. R. A. Brand. Improving the validity of hyperfine field distributions from magnetic alloys: the NORMOS program. *Nucl. Instrum. Methods Phys. Res. B*, 28:398–416, 1987.
4. R. A. Brand. *NORMOS Programs: SITE and DIST*, 1990. Program manual.
5. Z. Klencsar. MossWinn — methodological advances in the field of Mössbauer data analysis. *Hyperfine Interact.*, 217:117–126, 2013.
6. M. E. Matsnev and V. S. Rusakov. SpectrRelax: an application for Mössbauer spectra modeling and fitting. *AIP Conf. Proc.*, 1489:178–185, 2012.
7. S. Kamusella and H.-H. Klauß. Moessfit: a free Mössbauer fitting program. *Hyperfine Interact.*, 237:82, 2016.
8. K. Lagarec and D. G. Rancourt. RECOIL, Mössbauer spectral analysis software for Windows (version 1.0). *Department of Physics, University of Ottawa, Research Report*, 1998.
9. M. F. Hansen and S. Mørup. VINDA: a visual interactive Mössbauer data analysis program. Software package, Technical University of Denmark, 1999.
10. S. Margulies and J. R. Ehrman. Transmission and line broadening of resonance radiation incident on a resonance absorber. *Nucl. Instrum. Methods*, 12:131–137, 1961.
11. W. Kündig. Evaluation of the electric-field gradient and the asymmetry parameter η in Mössbauer spectra. *Nucl. Instrum. Methods*, 48:219–228, 1967.

12. M. Blume and J. A. Tjon. Mössbauer spectra in a fluctuating environment. *Phys. Rev.*, 165:446–456, 1968.
13. L. Néel. Théorie du trainage magnétique des ferromagnétiques en grains fins avec applications aux terres cuites. *Ann. Géophys.*, 5:99–136, 1949.
14. W. F. Brown. Thermal fluctuations of a single-domain particle. *Phys. Rev.*, 130:1677–1686, 1963.
15. J. Hesse and A. Rübartsch. Model independent evaluation of overlapped Mössbauer spectra. *J. Phys. E: Sci. Instrum.*, 7:526–532, 1974.
16. P. C. Hansen. Analysis of discrete ill-posed problems by means of the L-curve. *SIAM Rev.*, 34:561–580, 1992.
17. D. G. Rancourt and J. Y. Ping. Voigt-based methods for arbitrary-shape static hyperfine parameter distributions in Mössbauer spectroscopy. *Nucl. Instrum. Methods Phys. Res. B*, 58:85–97, 1991.
18. G. M. Kalvius and P. Kienle. The Mössbauer effect and its applications. *Z. Phys.*, 227:388–406, 1969.
19. W. A. Rolke, A. M. López, and J. Conrad. Limits and confidence intervals in the presence of nuisance parameters. *Nucl. Instrum. Methods Phys. Res. A*, 551:493–503, 2005.
20. R. D. Peng. Reproducible research in computational science. *Science*, 334:1226–1227, 2011.
21. J. Sánchez Marcos and N. Menéndez González. Fitbauer: software for Mössbauer spectrum fitting and analysis. <https://doi.org/10.5281/zenodo.21858360>, 2026.